

Name & Surname: Byron Broadway

Student Number: 05HA2312474

Name of your Support Centre: BELLVILLE



```
//
=====

// SABC Automated Licence Fee Management System (ALFMS) - Prototype
//
=====

// This prototype demonstrates the core functionality of ALFMS as proposed in the
// system design report. It simulates key modules including data integration,
// compliance monitoring, automated enforcement, communication, payment processing,
// and analytics dashboards.
//
// Technology: JavaScript (Node.js compatible)
// Author: Byron
// Student Number: 05HA2312474
// Module: HPXN302-1 Praxis N3B SA2 January - June 2026
//
=====

//
=====

// MODULE 1: DATA INTEGRATION LAYER (Registration & Account Management)
//
=====

// Simulates integration with SABC licence database, retail TV sales records,
// and Department of Home Affairs population register (Section 7.3 of report)

const sabcLicenceDB = [
```

```
{ id: "LIC001", idNumber: "8501015800083", name: "John Stemmet", paid: true, expiry: "2026-12-31", riskScore: 15 },  
  
{ id: "LIC002", idNumber: "9003025800085", name: "Thandi Nkosi", paid: false, expiry: "2025-06-30", riskScore: 78 },  
  
{ id: "LIC003", idNumber: "7812125800087", name: "Miriam Dlamini", paid: true, expiry: "2026-08-15", riskScore: 10 },  
  
];
```

```
const retailTVSalesDB = [  
  
  { idNumber: "8501015800083", name: "John Stemmet", tvPurchaseDate: "2024-03-15", retailer: "Game" },  
  
  { idNumber: "9003025800085", name: "Thandi Nkosi", tvPurchaseDate: "2023-11-20", retailer: "Makro" },  
  
  { idNumber: "7812125800087", name: "Miriam Dlamini", tvPurchaseDate: "2025-01-10", retailer: "HiFi Corp" },  
  
  { idNumber: "[CREDIT_DEBIT_CARD_NUMBER]", name: "Lerato Molefe", tvPurchaseDate: "2025-05-22", retailer: "Incredible Connection" },  
  
  { idNumber: "9505055800081", name: "Michael Tau", tvPurchaseDate: "2024-08-30", retailer: "Game" },  
  
  { idNumber: "7001015800082", name: "Dept of Education", tvPurchaseDate: "2024-02-14", retailer: "Makro" },  
  
];
```

```
const homeAffairsDB = [  
  
  { idNumber: "8501015800083", name: "John Stemmet", address: "12 Main Rd, Soweto", province: "Gauteng" },  
  
  { idNumber: "9003025800085", name: "Thandi Nkosi", address: "45 Church St, Pretoria", province: "Gauteng" },  
  
  { idNumber: "7812125800087", name: "Miriam Dlamini", address: "78 Long St, Cape Town", province: "Western Cape" },  
  
  { idNumber: "[CREDIT_DEBIT_CARD_NUMBER]", name: "Lerato Molefe", address: "23 Vilakazi St, Johannesburg", province: "Gauteng" },  
  
  { idNumber: "9505055800081", name: "Michael Tau", address: "9 Nelson Mandela Dr, Durban", province: "KwaZulu-Natal" },  
  
];
```

```
{ idNumber: "7001015800082", name: "Dept of Education", address: "222 Struben St, Pretoria",  
  province: "Gauteng" },
```

```
];
```

```
// Annual TV licence fee (R265 as per current SABC regulations)
```

```
const ANNUAL_LICENCE_FEE = 265;
```

```
//  
=====
```

```
// MODULE 3: COMPLIANCE MONITORING & RISK SCORING (Section 7.5 of report)
```

```
//  
=====
```

```
// Identifies non-compliant households by cross-referencing TV sales data with  
// the licence database. Assigns risk scores based on payment history and status.
```

```
function calculateRiskScore(person, licence) {
```

```
  let score = 0;
```

```
  // No licence at all = highest risk
```

```
  if (!licence) {
```

```
    score += 50;
```

```
  } else {
```

```
    // Unpaid licence
```

```
    if (!licence.paid) score += 35;
```

```
    // Expired licence
```

```
    if (new Date(licence.expiry) < new Date()) score += 25;
```

```
  }
```

```
  // Time since TV purchase (longer = higher risk of habitual non-compliance)
```

```
const purchaseDate = new Date(person.tvPurchaseDate);
const monthsSincePurchase = Math.floor((new Date() - purchaseDate) / (1000 * 60 * 60 * 24 * 30));
if (monthsSincePurchase > 12) score += 15;
if (monthsSincePurchase > 24) score += 10;

return Math.min(score, 100); // Cap at 100
}
```

```
function identifyNonCompliantHouseholds() {
  const nonCompliant = [];

  retailTVSalesDB.forEach((buyer) => {
    const licence = sabcLicenceDB.find((lic) => lic.idNumber === buyer.idNumber);

    if (!licence) {
      // TV owner with NO licence at all
      nonCompliant.push({
        idNumber: buyer.idNumber,
        name: buyer.name,
        status: "NO_LICENCE",
        tvPurchaseDate: buyer.tvPurchaseDate,
        retailer: buyer.retailer,
        riskScore: calculateRiskScore(buyer, null),
        segment: "Never Registered",
      });
    } else if (!licence.paid) {
      // Has licence but hasn't paid
      nonCompliant.push({
```

```
        idNumber: buyer.idNumber,
        name: buyer.name,
        status: "UNPAID",
        licenceId: licence.id,
        expiry: licence.expiry,
        riskScore: calculateRiskScore(buyer, licence),
        segment: "Lapsed",
    });
} else if (new Date(licence.expiry) < new Date()) {
    // Licence expired
    nonCompliant.push({
        idNumber: buyer.idNumber,
        name: buyer.name,
        status: "EXPIRED",
        licenceId: licence.id,
        expiry: licence.expiry,
        riskScore: calculateRiskScore(buyer, licence),
        segment: "Lapsed",
    });
}
});

// Sort by risk score (highest first) for prioritised enforcement
return nonCompliant.sort((a, b) => b.riskScore - a.riskScore);
}

//
=====
=====

// MODULE 4: AUTOMATED ENFORCEMENT & ESCALATION (Section 7.6 of report)
```

```
//
=====
=====

// Implements the 6-stage enforcement workflow: Reminder → Warning → Penalty →
// Debt Collection → Legal Action. Escalation is based on days overdue.

const ENFORCEMENT_STAGES = [
  { stage: 1, name: "Friendly Reminder", daysOverdue: 30, action: "SMS + Email" },
  { stage: 2, name: "Firm Reminder", daysOverdue: 60, action: "SMS + Email + Push
Notification" },
  { stage: 3, name: "Formal Warning", daysOverdue: 90, action: "Registered Letter" },
  { stage: 4, name: "Penalty Applied", daysOverdue: 120, action: "10% monthly penalty added" },
  { stage: 5, name: "Debt Collection", daysOverdue: 180, action: "Handover to external
collectors" },
  { stage: 6, name: "Legal Action", daysOverdue: 365, action: "Referral to SABC legal panel" },
];

function determineEnforcementStage(licence) {
  if (!licence || !licence.expiry) return ENFORCEMENT_STAGES[0];

  const daysOverdue = Math.floor((new Date() - new Date(licence.expiry)) / (1000 * 60 * 60 *
24));

  // Find the appropriate enforcement stage
  let currentStage = ENFORCEMENT_STAGES[0];
  for (const stage of ENFORCEMENT_STAGES) {
    if (daysOverdue >= stage.daysOverdue) {
      currentStage = stage;
    }
  }

  return currentStage;
}
```

```
}

function processEnforcement(nonCompliantList) {
  const enforcementActions = [];

  nonCompliantList.forEach((person) => {
    const licence = sabcLicenceDB.find((lic) => lic.idNumber === person.idNumber);
    const stage = determineEnforcementStage(licence);

    enforcementActions.push({
      name: person.name,
      idNumber: person.idNumber,
      riskScore: person.riskScore,
      enforcementStage: stage.stage,
      stageName: stage.name,
      action: stage.action,
      escalationDate: new Date(Date.now() + stage.daysOverdue * 24 * 60 * 60 * 1000)
        .toISOString()
        .split("T")[0],
    });
  });

  return enforcementActions;
}

//
=====
// MODULE 5: MULTI-CHANNEL COMMUNICATION HUB (Section 7.7 of report)
```



```
//  
=====
```

```
// Sends personalised notifications based on account status and enforcement stage.
```

```
// Supports SMS, Email, Push Notification, and WhatsApp channels.
```

```
function sendNotifications(nonCompliantList) {  
  const notifications = [];  
  
  nonCompliantList.forEach((person) => {  
    const contact = homeAffairsDB.find((h) => h.idNumber === person.idNumber);  
    const licence = sabcLicenceDB.find((lic) => lic.idNumber === person.idNumber);  
    const stage = determineEnforcementStage(licence);  
    let message = "";  
    let channel = "";  
  
    switch (person.status) {  
      case "NO_LICENCE":  
        message = `Dear ${person.name}, our records indicate you purchased a TV on  
${person.tvPurchaseDate} from ${person.retailer} but have no valid SABC TV licence. Please  
register and pay the annual fee of R${ANNUAL_LICENCE_FEE} at https://salfms.sabc.co.za  
within 30 days to avoid penalties.`;  
        channel = "SMS + Email";  
        break;  
      case "UNPAID":  
        message = `Dear ${person.name}, your TV licence (${person.licenceId}) remains unpaid.  
The annual fee is R${ANNUAL_LICENCE_FEE}. Please make payment at  
https://salfms.sabc.co.za to avoid escalation to Stage ${stage.stage + 1}:  
${ENFORCEMENT_STAGES[Math.min(stage.stage, 5)].name}`;  
        channel = "SMS + Email + Push";  
        break;  
      case "EXPIRED":
```

```
message = `Dear ${person.name}, your TV licence (${person.licenceId}) expired on
${person.expiry}. Please renew at https://salfms.sabc.co.za. Renewal fee:
R${ANNUAL_LICENCE_FEE}.`;
```

```
channel = "SMS + Email";
```

```
break;
```

```
}
```

```
notifications.push({
```

```
  recipient: person.name,
```

```
  address: contact ? contact.address : "Unknown",
```

```
  province: contact ? contact.province : "Unknown",
```

```
  channel: channel,
```

```
  enforcementStage: stage.name,
```

```
  message: message,
```

```
  sentDate: new Date().toISOString().split("T")[0],
```

```
  language: "English", // Default; system supports all 11 official languages
```

```
});
```

```
});
```

```
return notifications;
```

```
}
```

```
//
```

```
=====
```

```
// MODULE 2: INTELLIGENT BILLING & PAYMENT ENGINE (Section 7.4 of report)
```

```
//
```

```
=====
```

```
// Processes payments via multiple channels. Supports new registrations and renewals.
```

```
// Calculates early payment discounts and flexible payment plans.
```

```
function processPayment(idNumber, amount, paymentMethod = "Card") {
  const licence = sabcLicenceDB.find((lic) => lic.idNumber === idNumber);
  const paymentDate = new Date().toISOString().split("T")[0];

  // Apply 5% early payment discount for annual upfront payment
  let discount = 0;
  if (amount >= ANNUAL_LICENCE_FEE) {
    discount = ANNUAL_LICENCE_FEE * 0.05;
  }

  if (licence) {
    // Existing licence - renew
    licence.paid = true;
    licence.riskScore = Math.max(0, licence.riskScore - 20); // Reduce risk score on payment
    licence.expiry = new Date(new Date().setFullYear(new Date().getFullYear() + 1))
      .toISOString()
      .split("T")[0];

    return {
      success: true,
      transactionId: `TXN${Date.now()}`,
      message: `Payment of R${amount} received via ${paymentMethod}. Licence ${licence.id}
renewed until ${licence.expiry}.`,
      discount: discount > 0 ? `R${discount.toFixed(2)} early payment discount applied` : "None",
      newExpiry: licence.expiry,
      paymentDate: paymentDate,
    };
  } else {
```

```
// New licence registration + payment

const person = homeAffairsDB.find((h) => h.idNumber === idNumber);

const newLicence = {
  id: `LIC${String(sabcLicenceDB.length + 1).padStart(3, "0")}`,
  idNumber: idNumber,
  name: person ? person.name : "Unknown",
  paid: true,
  expiry: new Date(new Date().setFullYear(new Date().getFullYear() + 1))
    .toISOString()
    .split("T")[0],
  riskScore: 0,
};

sabcLicenceDB.push(newLicence);

return {
  success: true,
  transactionId: `TXN${Date.now()}`,
  message: `New licence ${newLicence.id} created for ${newLicence.name}. Valid until
${newLicence.expiry}.`,
  discount: discount > 0 ? `R${discount.toFixed(2)} early payment discount applied` : "None",
  newExpiry: newLicence.expiry,
  paymentDate: paymentDate,
};
}
}

// Payment plan calculator

function calculatePaymentPlan(frequency = "monthly") {
  const plans = {
```

```
    weekly: { amount: (ANNUAL_LICENCE_FEE / 52).toFixed(2), instalments: 52, frequency:
"Weekly" },

    fortnightly: { amount: (ANNUAL_LICENCE_FEE / 26).toFixed(2), instalments: 26, frequency:
"Fortnightly" },

    monthly: { amount: (ANNUAL_LICENCE_FEE / 12).toFixed(2), instalments: 12, frequency:
"Monthly" },

    quarterly: { amount: (ANNUAL_LICENCE_FEE / 4).toFixed(2), instalments: 4, frequency:
"Quarterly" },

    annual: { amount: ANNUAL_LICENCE_FEE.toFixed(2), instalments: 1, frequency: "Annual
(5% discount applied)", discountedAmount: (ANNUAL_LICENCE_FEE * 0.95).toFixed(2) },

};

return plans[frequency] || plans.monthly;
}
```

```
//
=====
=====

// MODULE 6: ANALYTICS, REPORTING & COMPLIANCE DASHBOARD (Section 7.8 of
report)

//
=====
=====

// Generates real-time compliance metrics, provincial breakdowns, and revenue projections.
```

```
function generateComplianceDashboard() {
    const totalTVOwners = retailTVSalesDB.length;
    const compliantOwners = retailTVSalesDB.filter((buyer) => {
        const licence = sabcLicenceDB.find((lic) => lic.idNumber === buyer.idNumber);
        return licence && licence.paid && new Date(licence.expiry) >= new Date();
    }).length;

    const complianceRate = ((compliantOwners / totalTVOwners) * 100).toFixed(1);
```

```
const nonCompliant = identifyNonCompliantHouseholds();  
const estimatedRevenueLoss = nonCompliant.length * ANNUAL_LICENCE_FEE;
```

```
// Provincial breakdown
```

```
const provincialCompliance = {};  
retailTVSalesDB.forEach((buyer) => {  
  const person = homeAffairsDB.find((h) => h.idNumber === buyer.idNumber);  
  const province = person ? person.province : "Unknown";  
  if (!provincialCompliance[province]) {  
    provincialCompliance[province] = { total: 0, compliant: 0 };  
  }  
  provincialCompliance[province].total++;  
  const licence = sabcLicenceDB.find((lic) => lic.idNumber === buyer.idNumber);  
  if (licence && licence.paid && new Date(licence.expiry) >= new Date()) {  
    provincialCompliance[province].compliant++;  
  }  
});
```

```
return {  
  reportDate: new Date().toISOString().split("T")[0],  
  totalTVOwners,  
  compliantOwners,  
  nonCompliantOwners: nonCompliant.length,  
  complianceRate: `${complianceRate}%`,  
  estimatedAnnualRevenueLoss: `R${estimatedRevenueLoss.toLocaleString()}`,  
  nonCompliantBreakdown: {  
    noLicence: nonCompliant.filter((p) => p.status === "NO_LICENCE").length,  
    unpaid: nonCompliant.filter((p) => p.status === "UNPAID").length,  
    expired: nonCompliant.filter((p) => p.status === "EXPIRED").length,
```

```
    },  
    provincialCompliance: provincialCompliance,  
    projectedRevenueIfCompliant: `R${(totalTVOwners *  
ANNUAL_LICENCE_FEE).toLocaleString()}`,  
  };  
}
```

```
//  
=====
```

```
// SYSTEM EXECUTION
```

```
//  
=====
```

```
console.log("=====");  
console.log("|| SABC Automated Licence Fee Management System (SALFMS) ||");  
console.log("|| Prototype Demonstration ||");  
console.log("=====");  
console.log("");
```

```
// Step 1: Identify non-compliant households (Module 3)
```

```
console.log("———— STEP 1: Compliance Monitoring & Risk Scoring ————");  
const nonCompliant = identifyNonCompliantHouseholds();  
console.log(`Found ${nonCompliant.length} non-compliant TV owner(s) (sorted by risk score):`);  
nonCompliant.forEach((p) =>  
  console.log(` • ${p.name} | Status: ${p.status} | Risk Score: ${p.riskScore}/100 | Segment:  
${p.segment}`)  
);
```

// Step 2: Determine enforcement actions (Module 4)

```
console.log("\n———— STEP 2: Automated Enforcement Escalation ————");

const enforcement = processEnforcement(nonCompliant);

enforcement.forEach((e) =>

  console.log(` • ${e.name} → Stage ${e.enforcementStage}: ${e.stageName} (Action:
  ${e.action})`))

);
```

// Step 3: Send automated notifications (Module 5)

```
console.log("\n———— STEP 3: Multi-Channel Communication ————");

const notifications = sendNotifications(nonCompliant);

notifications.forEach((n) =>

  console.log(` → [${n.channel}] To: ${n.recipient} (${n.province})\n  "${n.message}"\n`))

);
```

// Step 4: Simulate a payment (Module 2)

```
console.log("———— STEP 4: Payment Processing ————");

const payment = processPayment("[CREDIT_DEBIT_CARD_NUMBER]",
ANNUAL_LICENCE_FEE, "Instant EFT");

console.log(` ✓ ${payment.message}`);

console.log(` Transaction ID: ${payment.transactionId}`);

console.log(` Discount: ${payment.discount}`);
```

// Step 5: Show payment plan options

```
console.log("\n———— STEP 5: Flexible Payment Plans ————");

const plans = ["weekly", "fortnightly", "monthly", "quarterly", "annual"];

plans.forEach((freq) => {

  const plan = calculatePaymentPlan(freq);

  const display = plan.discountedAmount
```


